Chương IV

TÌM KIẾM HEURISTIC

Nội dung chính: Tiếp theo các chiến lược tìm kiếm hình thức trong không gian trạng thái, chương này giới thiệu các chiến lược tìm kiếm mang tính không hình thức – tìm kiếm heuristic. Không gian tìm kiếm của các bài toán luôn có xu hướng tăng lên theo hàm mũ, nên tìm kiếm heuristic là một công cụ chủ yếu để xử lý sự bùng nổ tổ hợp này. Nội dung chương IV giới thiệu hai thuật toán heuristic cơ bản là: tìm kiếm *tốt nhất đầu tiên* (best first search) và tìm kiếm *leo núi* (hill climbing), sau đó chú trọng vào việc phân tính hành vi của các thuật toán heuristic trên không gian, xem xét các đặc tính có thể chấp nhận được, tính đơn nhất và khả năng cung cấp thông tin của một heuristic.

Mục tiêu cần đạt : Sau chương này, sinh viên có thể :

- Hiểu khái niệm và nguyên tắc áp dụng heuristic vào việc tìm kiếm trong không gian trạng thái.
- Vận dụng heuristic vào một số bài toán phổ biến.
- Vận dụng các chiến lược tìm kiếm heuristic vào các bài toán trò chơi.
- Phân tích các heuristic khác nhau có thể áp dụng cho bài toán.

Kiến thức tiên quyết : Lý thuyết đồ thị, Các thuật toán tìm kiếm trên đồ thị, Lý thuyết trò chơi, …

Tài liệu tham khảo :

[1] George F. Luger, William A. Stubblefield – Albuquerque – *Artificial Intelligence* – Wesley Publishing Company, Inc – 1997 ***(Chapter 4)***

[2] Bùi Xuân Toại – Trương Gia Việt (Biên dịch) – *Trí tuệ nhân tạo – Các cấu trúc và chiến lược giải quyết vấn đề* - NXB Thống kê, 2000 (Phần II)

[3] *Heuristic search*
http://www.macs.hw.ac.uk/~alison/ai3notes/subsection2_6_2_3.html

[4] *Minimax and alpha-beta template*
http://www.cs.caltech.edu/~petrovic/games/archex/othellodir/node2.html

[5] Nicky Danino - September 3rd 2001- *Heuristic evaluation : A step by step Guide*
http://www.sitepoint.com/article/heuristic-evaluation-guide

I MỞ ĐẦU

George Polya định nghĩa heuristic là "sự nghiên cứu về các phương pháp và các qui tắc trong việc khám phá và phát minh" (Polya 1945). Nghĩa này có thể xuất phát từ gốc Hy Lạp của động từ eurisco nghĩa là "tôi phát hiện". Khi Archimedes nhảy ra khỏi bồn tắm và chộp lấy chiếc mũ miện bằng vàng, ông ta đã la lên "Eureka!" có nghĩa "Tôi đã tìm thấy nó!". Trong tìm kiếm không gian trạng thái, heuristic là các luật dùng để chọn những nhánh nào có nhiều khả năng nhất dẫn đến một giải pháp chấp nhận được.

Các chương trình giải quyết những vấn đề trí tuệ nhân tạo sử dụng heuristic cơ bản theo hai dạng:

1. Vấn đề có thể không có giải pháp chính xác vì những điều không rõ ràng trong diễn đạt vấn đề hoặc trong các dữ liệu có sẵn. Chẩn đoán y khoa là một ví dụ. Tập hợp các triệu chứng cho trước có thể do nhiều nguyên nhân gây ra, bác sĩ có thể dùng heuristic để chọn kết quả chẩn đoán nào thích hợp nhất và đưa ra kế hoạch điều trị.

2. Vấn đề có thể có giải pháp chính xác, nhưng chi phí tính toán để tìm ra nó không cho phép. Trong nhiều vấn đề (như cờ vua chẳng hạn), không gian trạng thái phát triển rất nhanh và rất rộng vì số lượng các trạng thái có thể xảy ra tăng theo hàm mũ hoặc giai thừa cùng với độ sâu tìm kiếm. Trong những trường hợp này, các kỹ thuật tìm kiếm thô sơ như tìm kiếm sâu hay tìm kiếm rộng sẽ không tìm được giải pháp trong một giới hạn thời gian. Heuristic sẽ giảm bớt độ phức tạp bằng cách hướng việc tìm kiếm theo con đường có nhiều hứa hẹn nhất. Nhờ đã loại bỏ bớt các trạng thái không hứa hẹn và con cháu của chúng ra khỏi việc xem xét nên thuật toán heuristic có thể khắc phục việc bùng nổ trạng thái và tìm ra một giải pháp có thể chấp nhận được.

Giống như tất cả các luật khám phá và phát minh khác, heuristic có thể sai lầm. Heuristic chỉ là một phỏng đoán chứa các thông tin về bước tiếp theo sẽ được chọn dùng trong việc giải quyết một vấn đề. Nó thường dựa vào kinh nghiệm hoặc trực giác. Vì các heuristic sử dụng những thông tin hạn chế nên chúng ít khi có khả năng đoán trước chính xác cách hành xử của không gian trạng thái ở những giai đoạn xa hơn. Heuristic có thể dẫn đến một thuật toán tìm kiếm chỉ đạt được giải pháp gần tối ưu hoặc hoàn toàn không tìm được bất kỳ giải pháp nào. Đây là một hạn chế thuộc về bản chất tìm kiếm heuristic.

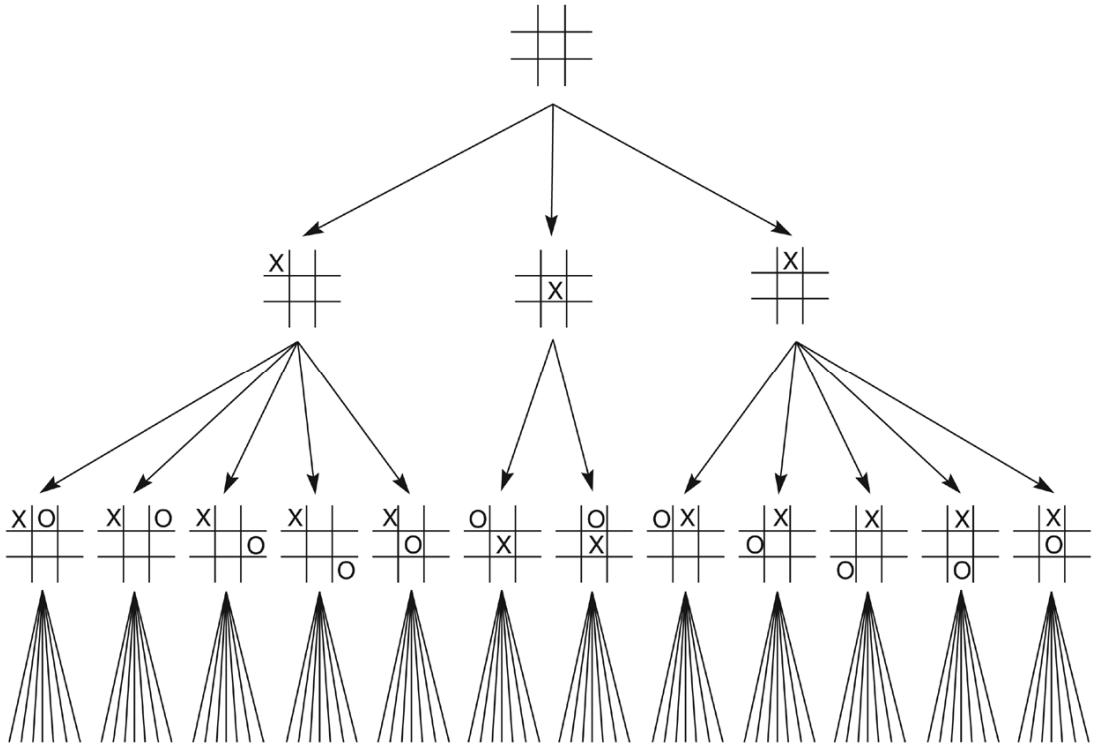
Các heuristic và việc thiết kế thuật toán để thực hiện tìm kiếm heuristic từ lâu đã là sự quan tâm chủ yếu của các công trình nghiên cứu trí tuệ nhân tạo. Chơi game và chứng minh định lý là hai ứng dụng lâu đời nhất, cả hai đều cần đến các heuristic để thu giảm bớt không gian giải pháp có thể. Không thể nào kiểm tra hết mọi suy luận có thể sinh ra trong lĩnh vực toán hoặc mọi nước đi có thể có trên bàn cờ vua, tìm kiếm heuristic thường là câu trả lời thực tế duy nhất. Gần đây việc tìm kiếm trong các hệ chuyên gia cũng xác nhận mức độ quan trọng của các heuristic như là một phần không thể thiếu trong quá trình giải quyết vấn đề.

Thuật toán heuristic gồm hai phần: Hàm đánh giá heuristic và thuật toán để sử dụng nó trong tìm kiếm không gian trạng thái.

Xét trò chơi Tic-tac-toe, để tìm kiếm đến hết không gian, số lượng trạng thái sẽ là rất lớn nhưng không phải là không thể vượt qua. Mỗi nước đi trong chín nước đầu tiên đều có tám khả năng đặt quân cờ kế tiếp và đến lượt mình mỗi nước đi này lại có bảy khả năng đặt quân

cờ cho nước đi tiếp tục ... Một phân tích đơn giản cho biết số lượng các trạng thái cần được xem xét cho quá trình này là 9 x 8 x 7 x ... x 1 = 9!.

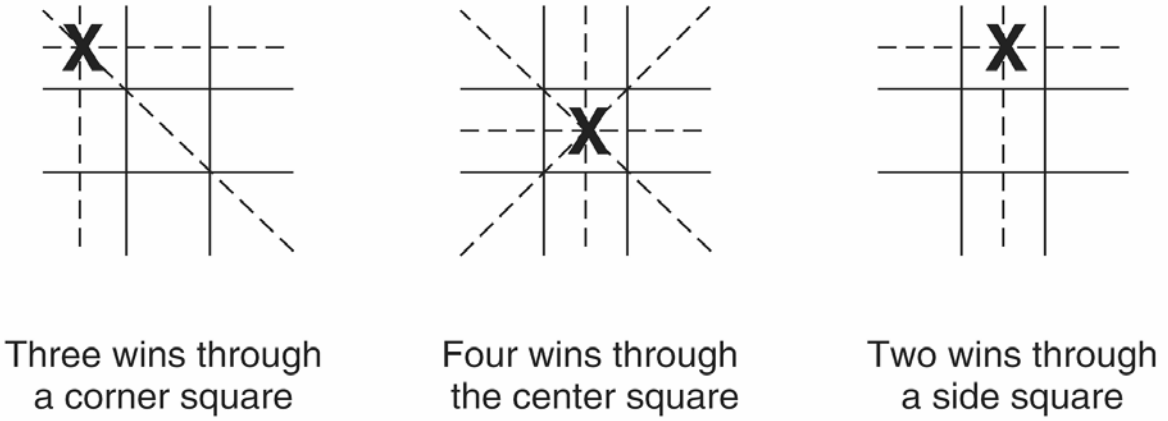
Áp dụng một nhận xét trực quan nhỏ dựa theo tính chất đối xứng của cấu hình bàn cờ có thể làm giảm nhỏ không gian tìm kiếm xuống một ít. Nhiều cấu hình của bài toán tương đương nhau trong các thao tác. Chẳng hạn, thực tế chỉ có ba nước đi cho quân cờ đầu tiên: ô cạnh, ô góc hoặc ô giữa. Các rút gọn đối xứng ở mức thứ hai của các trạng thái sẽ giảm tiếp số lượng đường đi có thể xảy ra trong không gian đó xuống đến tổng số 12 x 7! . Nó đã nhỏ hơn không gian ban đầu nhưng vẫn phát triển theo hàm giai thừa.



Hình 4.1 – *Không gian trạng thái bài toán Tic-tac-toe thu giảm bởi tính đối xứng*

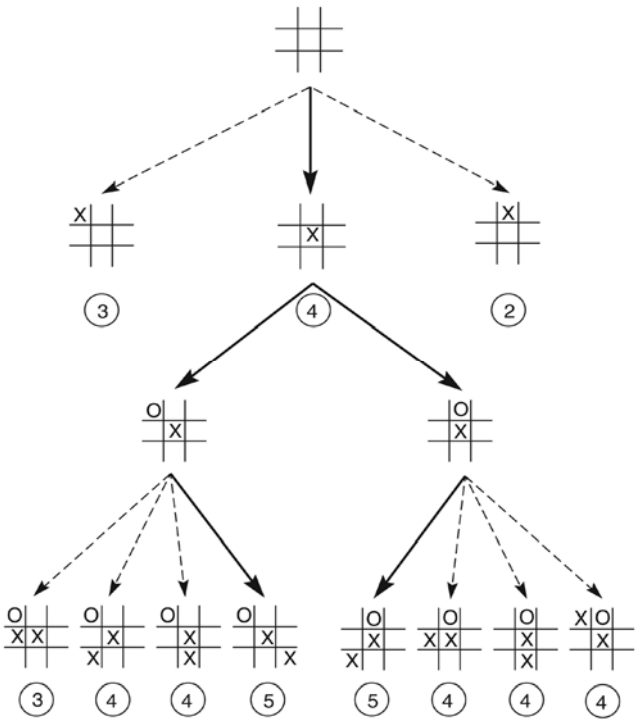
Tuy nhiên, một heuristic đơn giản có thể loại bỏ việc tìm kiếm hầu như toàn bộ: heuristic "nước đi chắc thắng nhất", nghĩa là chọn vị trí đặt quân cờ mà có nhiều đường chắc thắng nhất giao nhau. Trong trường hợp các trạng thái đều có số lượng bằng nhau, chọn trạng thái đầu tiên.

Sau đó thuật toán này sẽ chọn lựa và chuyển đến trạng thái có giá trị heuristic cao nhất (xem hình). Như trong hình vẽ, quân X sẽ chọn đặt vào vị trí trung tâm bàn cờ. Chú ý không chỉ các trạng thái tương đương khác bị loại bỏ mà tất cả con cháu của chúng cũng bị loại. Hai phần ba không gian trạng thái này đã bị loại ngay từ nước đi đầu tiên.



Hình 4.2 - *Heuristic "nước đi chắc thắng nhất"*

Sau nước đi đầu tiên, đối thủ có thể chọn một trong hai nước đi tương đương nhau. Dù chọn nước đi nào, heuristic đó cũng được áp dụng cho các bước tiếp theo. Khi quá trình tìm kiếm tiếp tục, từng bước đi sẽ đánh giá các con của một nút duy nhất mà không yêu cầu tìm kiếm hết không gian. Mặc dù khó tính chính xác số lượng trạng thái phải được kiểm tra theo cách này nhưng vẫn có thể tính phỏng đoán một giới hạn trên. Trong thực tế số lượng sẽ ít hơn 9! rất nhiều.



Hình 4.3 – *Không gian trạng thái đã được thu giảm bởi heuristic*

II THUẬT TOÁN TÌM KIẾM HEURISTIC

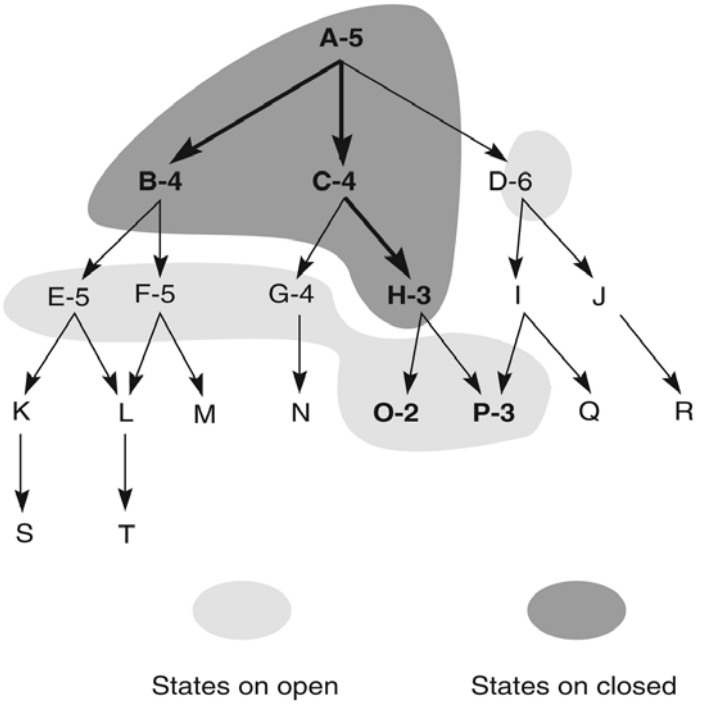
II.1 Tìm kiếm leo núi (Hill climbing – Pearl 1984)

Cách đơn giản nhất để thực hiện tìm kiếm heuristic là tìm kiếm "leo núi". Chiến lược leo núi phát triển trạng thái con tốt nhất sẽ được chọn cho bước tiếp theo, không lưu giữ lại bất kỳ thông tin nào về các nút anh em lẫn cha mẹ của nó. Quá trình tìm kiếm sẽ dừng lại khi tiếp cận trạng thái tốt hơn so với mọi trạng thái con của nó. Hình dung một người leo núi hăm hở nhưng mù quáng luôn luôn chọn leo lên đỉnh theo con đường dốc nhất có thể có cho đến khi không còn leo tiếp được nữa. Vì không ghi lại thông tin của quá trình đã xảy ra nên thuật toán này không thể phục hồi lại từ những thất bại trong chiến lược của nó.

Hạn chế chủ yếu của chiến lược leo núi là có xu hướng rơi vào "một cực đại cục bộ". Khi đến được một trạng thái tốt hơn so với mọi trạng thái con của nó, thuật toán dừng lại. Nếu trạng thái này không phải là đích mà chỉ là một điểm cực đại cục bộ, thuật toán sẽ thất bại trong việc tìm lời giải. Như vậy hiệu quả hoạt động chỉ có thể được cải thiện trong một phạm vi giới hạn nào đó, nhưng trong toàn bộ không gian có thể không bao giờ đạt được sự tối ưu tổng thể.

II.2 Tìm kiếm tốt nhất đầu tiên (Best – first – search)

Xét đồ thị không gian tìm kiếm như hình dưới đây (con số cạnh mỗi nút cho biết giá trị ước lượng độ tốt của nút đó trong không gian, giá trị thấp nhất là tốt nhất). Giả sử nút đích cần tìm kiếm là P.



***Hình 4.4** – Đồ thị cho giải thuật tìm kiếm tốt nhất đầu tiên*

Câu hỏi :



Trình bày danh sách các nút được duyệt dùng thuật toán leo núi cho đồ thị trong hình 4.4 ?

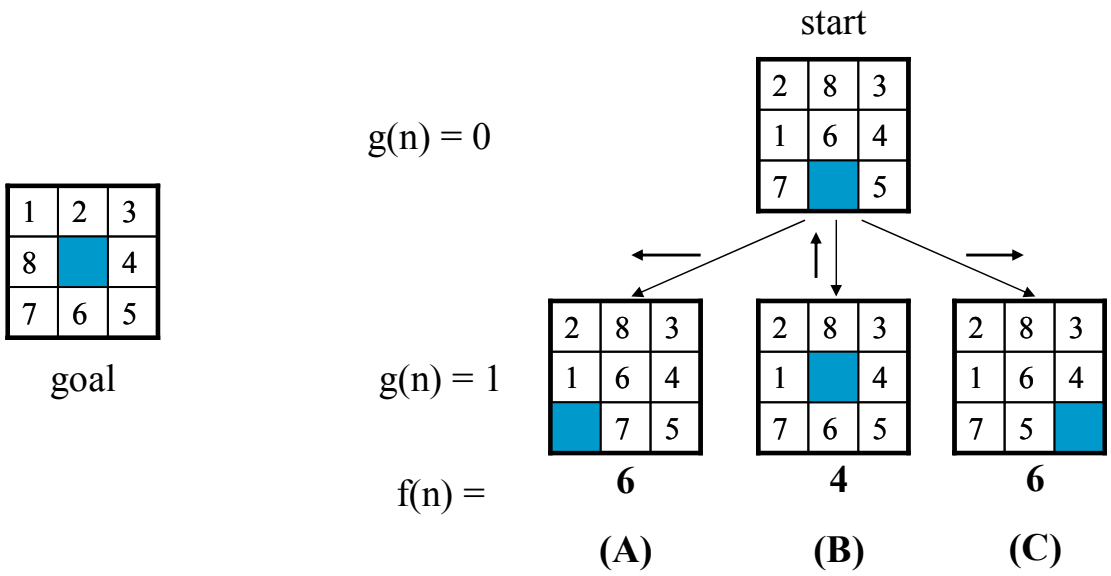
Giống như các thuật toán tìm kiếm sâu và rộng, tìm kiếm tốt nhất cũng dùng các danh sách để lưu giữ trạng thái: danh sách open chứa các nút được triển khai trong quá trình tìm kiếm và danh sách closed chứa các nút đã xét. Một bước mới được bổ sung vào thuật toán là sắp xếp các trạng thái trong danh sách open phù hợp với giá trị heuristic ước lượng "độ tốt" của chúng so với đích. Như vậy mỗi bước lặp của vòng lặp sẽ xem xét trạng thái "có hứa hẹn nhất" trong danh sách open và loại bỏ trạng thái này ra khỏi open. Nếu gặp trạng thái đích, thuật toán này sẽ cung cấp con đường lời giải đã dẫn đến đích đó. Nếu ngược lại, phần tử đầu tiên của open không phải là đích, thuật toán sẽ áp dụng các luật phù hợp để phát sinh con cháu. Trường hợp một trạng thái con nào đó đã có sẵn trong open hoặc closed, thuật toán cũng sẽ kiểm tra để chắc chắn rằng sẽ chọn được nút cung cấp con đường lời giải ngắn hơn. Các trạng thái lặp hai lần sẽ không được giữ lại. Nhờ cập nhật kịp thời nguồn gốc của các nút trong open và closed nên thuật toán này có nhiều khả năng tìm được đường đi ngắn nhất dẫn đến đích. Khi open được duy trì dưới dạng một danh sách có sắp xếp, nó thường được tổ chức như là một *hàng ưu tiên* (Priority queue). Dưới đây trình bày các bước áp dụng thuật toán tìm kiếm cho đồ thị trong hình trên.

1. open = [A5]; closed = []
2. Đánh giá A5;
   open = [B4,C4,D6];
   closed = [A5]
3. Đánh giá B4;
   open = [C4,E5,F5,D6];
   closed = [B4,A5]
4. Đánh giá C4;
   open = [H3,G4,E5,F5,D6];
   closed = [C4,B4,A5]
5. Đánh giá H3;
   open = [O2,P3,G4,E5,F5,D6];
   closed = [H3,C4,B4,A5]
6. Đánh giá O2;
   open = [P3,G4,E5,F5,D6];
   closed = [O2,H3,C4,B4,A5]
7. Đánh giá P3;
   Tìm được lời giải!

II.3 Cài đặt hàm đánh giá heuristic (heuristic evaluation function)

Bây giờ ta đánh giá hiệu quả của vài heuristic khác nhau được dùng để giải trò đố 8 ô. Hình dưới đây trình bày trạng thái xuất phát và trạng thái đích của trò chơi cùng với ba trạng thái đầu tiên trong quá trình tìm kiếm.

Heuristic đơn giản nhất sẽ đếm *số ô sai khác so với trạng thái đích* trong từng trạng thái. Trạng thái có số ô sai khác ít nhất sẽ gần đích hơn và là trạng thái tốt nhất để kiểm tra kế tiếp.



Hình 4.5 *– Trạng thái bắt đầu và kết thúc trong trò đố 8 ô*

Tuy nhiên heuristic này không sử dụng hết các thông tin trong một cấu hình bàn cờ vì nó không đưa vào khoảng cách mà các ô sai khác. Một heuristic "tốt hơn" là sẽ cộng tất cả các khoảng cách đó lại thành *tổng số ô mà một ô phải di chuyển về vị trí đúng* của nó trong trạng thái đích (khoảng cách Manhattan).

Cả hai heuristic này đều có hạn chế là không thể biết rõ những khó khăn khi đổi chỗ hai ô. Đó là trường hợp hai ô nằm cạnh nhau và vị trí đúng của chúng là phải đổi chỗ cho nhau, ta phải mất nhiều (chứ không phải hai) nước đi mới đặt chúng lại được đúng vị trí. Một heuristic muốn tính toán điều này phải nhân ít nhất là gấp đôi khoảng cách đối với mỗi trường hợp có hai ô đổi chỗ trực tiếp.

Heuristic "khoảng cách Manhattan" cho chúng ta một dự đoán có vẻ chính xác hơn so với heuristic số ô sai khác so với trạng thái đích. Mục đích của chúng ta là dùng những thông tin hạn chế có sẵn trong một mô tả trạng thái để đưa ra những chọn lựa thông minh. Việc thiết kế các heuristic tốt là một vấn đề mang tính kinh nghiệm, óc phán đoán và trực giác, nhưng giá trị cuối cùng của một heuristic phải được đo bằng hiệu quả thực sự của nó trong từng tình huống bài toán.

Vì heuristic có thể sai lầm nên có khả năng thuật toán tìm kiếm sẽ dẫn đến một con đường không đưa đến đích. Vấn đề này đã xuất hiện trong tìm kiếm sâu, ở đó một giới hạn độ sâu đã được sử dụng để phát hiện những con đường thất bại. Ý tưởng này cũng có thể áp dụng

cho tìm kiếm heuristic. Nếu hai trạng thái có giá trị heuristic bằng nhau thì nên kiểm tra trạng thái nào gần trạng thái gốc của đồ thị hơn. Trạng thái gần gốc hơn sẽ có nhiều khả năng là con đường ngắn nhất dẫn đến đích. Khoảng cách từ trạng thái xuất phát có thể đo được bằng cách duy trì một số đếm chiều sâu cho từng trạng thái đếm. Số đếm này bằng 0 đối với trạng thái xuất phát và tăng lên một đơn vị sau mỗi mức tìm kiếm. Nó ghi lại độ dời thực tế phải thực hiện để đi từ trạng thái xuất phát đến từng trạng thái. Số đếm này có thể cộng thêm vào giá trị heuristic của từng trạng thái để hướng việc tìm kiếm theo khuynh hướng chọn những trạng thái gần trạng thái xuất phát hơn trong đồ thị.

Do đó hàm đánh giá f sẽ bao gồm tổng của hai phần:

$$f(n) = g(n) + h(n)$$

trong đó g(n) đo chiều dài thực từ trạng thái n bất kỳ về trạng thái xuất phát và h(n) là ước lượng heuristic cho khoảng cách từ trạng thái n đến trạng thái đích.

Chẳng hạn, trong bài toán trò đố 8 ô trên, chúng ta có thể gọi h(n) là số ô cần phải dời chỗ. Khi hàm đánh giá này được áp dụng cho từng trạng thái con (A), (B), (C) trong hình 4.5, các giá trị cho hàm f lần lượt là 6, 4 và 6.

Một heuristic dùng hàm đánh giá f(n) như trên kết hợp với thuật toán tìm kiếm tốt nhất đầu tiên best-first-search, được gọi là *thuật toán A* (algorithm A)

Câu hỏi :



Mật mã Caesar là một cách mã hóa đơn giản dựa vào phép hoán vị vòng tròn các chữ trong bảng chữ cái, với chữ cái thứ i được thay thế bởi chữ cái thứ (i + 1). Ví dụ, trong mật mã Caesar dịch 4 bậc, từ "Ceasar" sẽ được mã hóa thành "Geiwev". Nêu một heuristic mà bạn nghĩ có thể dùng để giải các mật mã Ceasar ?

II.4 Tính khả chấp, tính đơn nhất và khả năng cung cấp thông tin của heuristic

Có thể chúng ta phải đánh giá cách hành xử của các heuristic trên một số phương diện. Ví dụ, có thể chúng ta không chỉ cần một giải pháp mà còn cần thuật toán để tìm con đường ngắn nhất dẫn đến đích. Một số tính chất dưới đây là cần xem xét đối với việc đánh giá hiệu quả của một heuristic :

II.4.1 Tính khả chấp :

Một heuristic dùng để tìm ra con đường dẫn đến đích ngắn nhất bất cứ khi nào nó có tồn tại được gọi là heuristic khả chấp (admissible). Nói cách khác, tính khả chấp của heuristic là nó sẽ bảo đảm tìm thấy đường đi ngắn nhất đến trạng thái đích.

Một thuật toán tìm kiếm có thể chấp nhận được nếu nó được đảm bảo sẽ tìm thấy một đường đi tối thiểu dẫn đến lời giải, bất kỳ lúc nào con đường đó có mặt. Trong việc xác định tính khả chấp của một heuristic, chúng ta định nghĩa hàm đánh giá f* :

$$f^*(n) = g^*(n) + h^*(n)$$

Với g*(n) là giá của đường đi ngắn nhất từ nút bắt đầu đến nút n, còn h*(n) là giá **thực sự** của đường đi ngắn nhất từ nút n đến nút mục tiêu. Như vậy f*(n) là chi phí thực của con đường tối ưu từ nút xuất phát đến nút đích đi qua nút n.

Nếu thuật toán A dùng hàm đánh giá f, trong đó $h(n) \le h^*(n)$ thì nó được gọi là *thuật toán A**(algorithm A)

II.4.2 Tính đơn nhất :

Khi có một trạng thái được phát hiện nhờ sử dụng tìm kiếm heuristic, liệu có bảo đảm rằng về sau sẽ không tìm được một trạng thái như vậy với khoảng cách ngắn hơn tính từ trạng thái xuất phát. Đây chính là thuộc tính của sự đơn nhất (monotocinity). Nói cách khác, tính đơn nhất của một heuristic là nó sẽ bảo đảm đường đi ngắn nhất đến mỗi trạng thái.

Một heuristic h sẽ là đơn nhất nếu :

- Đối với tất cả các trạng thái n và n+1, ta có :

  $h(n) - h(n+1) \le cost(n, n+1)$,

trong đó cost(n, n+1) là chi phí thực tính của đường đi từ trạng thái n đến trạng thái n+1.

- Giá trị heuristic của trạng thái đích là 0, tức h(goal) = 0.

I.1.1. Khả năng cung cấp thông tin :

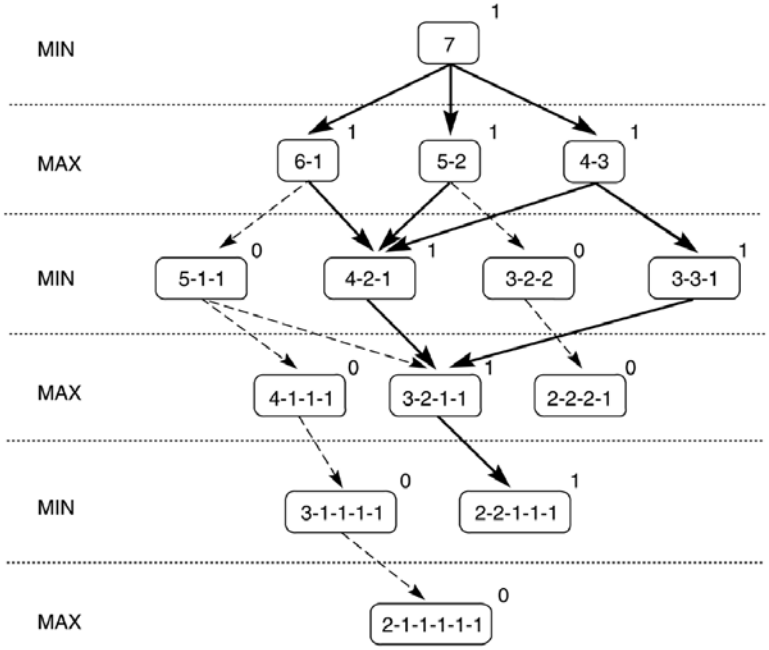
Chúng ta có thể đặt câu hỏi liệu có một heuristic nào "tốt hơn" những cái khác hay không? Heuristic này tốt hơn heuristic kia theo ý nghĩa nào ? Đây là khả năng cung cấp thông tin (informedness) của một heuristic.

Đối với hai heuristic h_1 và h_2, nếu $h_1(n) \le h_2(n)$ ứng với tất cả các trạng thái n trong không gian tìm kiếm thì heuristic h_2 được gọi là có khả năng cung cấp thông tin nhiều hơn so với h_1.

III SỬ DỤNG HEURISTIC TRONG CÁC TRÒ CHƠI

III.1 Thủ tục minimax

Xét các trò chơi hai đối thủ đối kháng, chẳng hạn trò chơi *nim*. Để chơi nim, một số token (vật biểu hiện như đồng xu, lá bài, mảnh gỗ, ...) được đặt trên bàn giữa hai đối thủ. Ở mỗi nước đi, người chơi phải chia đống token thành hai đống nhỏ có số lượng khác nhau. Ứng với một số token vừa phải, không gian trạng thái này có thể triển khai đến cùng. Hình sau biểu diễn không gian trạng thái của trò chơi có 7 token.



Hình 4.6 *– Không gian trạng thái của trò chơi nim*

Khi chơi các trò chơi có thể triển khai hết không gian trạng thái, khó khăn chủ yếu là phải tính toán phản ứng của đối thủ. Một cách xử lý đơn giản nhất là giả sử đối thủ của bạn cũng sử dụng kiến thức về không gian trạng thái giống như bạn và áp dụng kiến thức đó kiên định để thắng cuộc. Mặc dù giả thiết này có những hạn chế của nó nhưng nó cũng cho chúng ta một cơ sở hợp lý để dự đoán hành vi của đối thủ. Minimax sẽ tìm kiếm không gian của trò chơi này theo giả thiết đó.

Hai đối thủ trong một trò chơi được gọi là MIN và MAX. MAX đại diện cho đối thủ quyết giành thắng lợi hay cố gắng tối đa hóa ưu thế của mình. Ngược lại MIN là đối thủ cố gắng tối thiểu hóa điểm số của MAX. Ta giả thiết MIN cũng dùng cùng những thông tin như MAX.

Khi áp dụng thủ tục Minimax, chúng ta đánh dấu luân phiên từng mức trong không gian tìm kiếm phù hợp với đối thủ có nước đi ở mức đó. Trong ví dụ trên, MIN được quyền đi trước, từng nút lá được gán giá trị 1 hay 0 tùy theo kết quả đó là thắng cuộc đối với MAX hay MIN. Minimax sẽ truyền các giá trị này lên cao dần trên đồ thị qua các nút cha mẹ kế tiếp nhau theo luật sau:

- Nếu trạng thái cha mẹ là nút MAX, gán cho nó giá trị tối đa của các con cháu của nó.

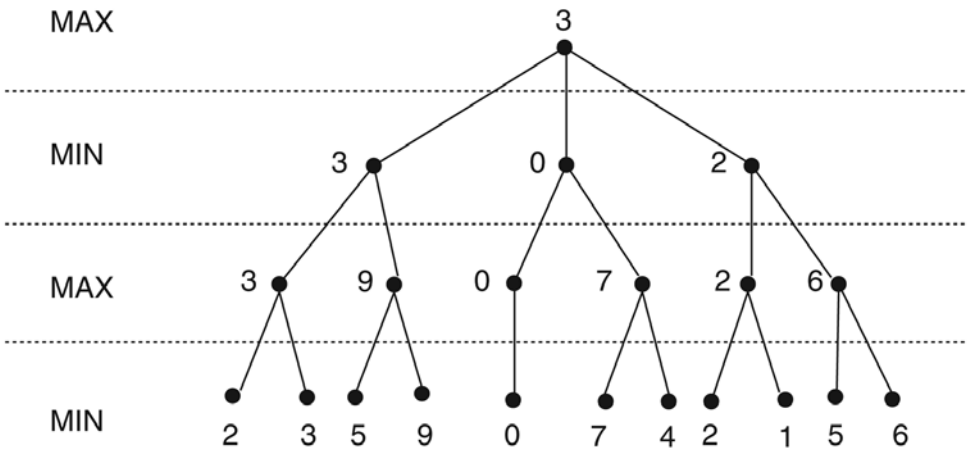
- Nếu trạng thái cha mẹ là nút MIN, gán cho nó giá trị tối thiểu của các con cháu của nó.

Giá trị được gán cho từng trạng thái bằng cách đó sẽ chỉ rõ giá trị của trạng thái tốt nhất mà đối thủ này có thể hy vọng đạt được. Các giá trị này sẽ được dùng để lựa chọn các nước đi có thể có. Kết quả của việc áp dụng Minimax vào đồ thị không gian trạng thái đối với trò chơi Nim được thể hiện như hình trên. Vì tất cả các nước đi đầu tiên có thể xảy ra cho MIN sẽ dẫn đến các nút có giá trị 1 nên đối thủ MAX luôn có thể bắt trò chơi giành thắng lợi cho mình bất kể nước đi đầu tiên của MIN là như thế nào (đường đi thắng lợi của MAX được cho theo mũi tên đậm).

III.2 Áp dụng minimax đến độ sâu lớp cố định

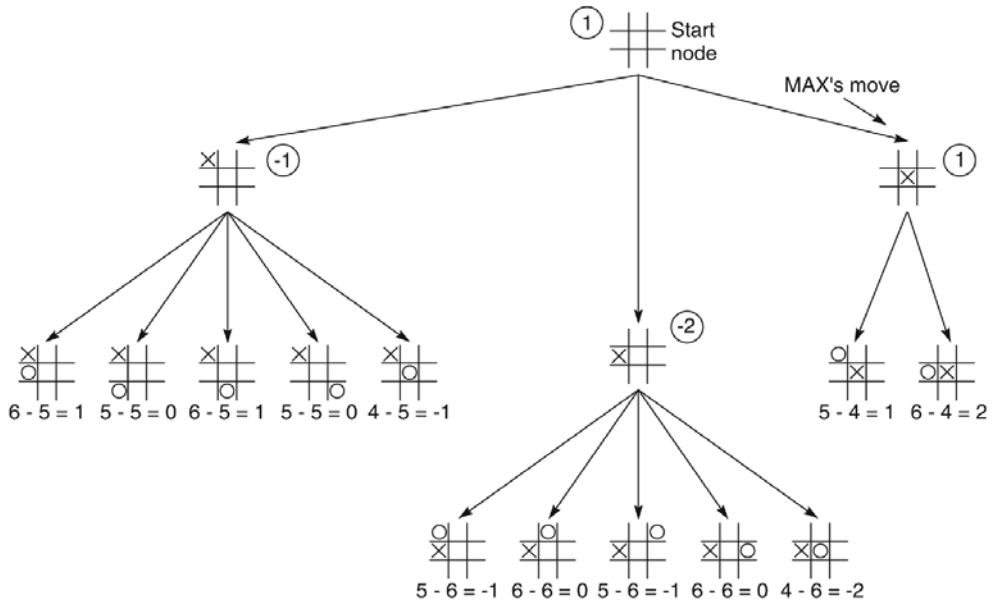
Khi áp dụng Minimax cho các trò chơi phức tạp, hiếm khi có khả năng mở rộng đồ thị không gian trạng thái đến các nút lá. Thay vào đó không gian trạng thái này chỉ có thể được triển khai đến một số mức xác định phụ thuộc tiềm năng về thời gian và bộ nhớ chẳng hạn. Chiến lược này được gọi là *tính trước n nước đi* (n –move lookahead). Vì giá trị các nút trong đồ thị con này không phải là trạng thái kết thúc của trò chơi nên chúng không phản ánh giá trị thắng cuộc hay thua cuộc. Chúng chỉ có thể được gán một giá trị phù hợp với một hàm đánh giá heuristic nào đó. Giá trị được truyền ngược về nút gốc không cung cấp thông tin thắng cuộc hay thua cuộc mà chỉ là giá trị heuristic của trạng thái tốt nhất có thể tiếp cận sau n nước đi kể từ nút xuất phát. Việc tính trước này sẽ làm tăng hiệu quả của heuristic vì nó được áp dụng vào một phạm vi lớn hơn trong không gian trạng thái. Minimax sẽ hợp nhất tất cả các giá trị của các nút con cháu của một trạng thái thành một giá trị duy nhất cho trạng thái đó.

Trong các đồ thị trò chơi được tìm kiếm bằng mức hay lớp, MAX và MIN luân phiên nhau chọn các nước đi. Mỗi nước đi của một đối thủ sẽ xác định một lớp mới trên đồ thị. Các chương trình trò chơi nói chung đều dự tính trước một *độ sâu lớp cố định* (thường được xác định bằng các giới hạn về không gian hoặc thời gian của máy tính). Các trạng thái trên mức đó được đánh giá theo các heuristic và các giá trị này sẽ được truyền ngược lên bằng thủ tục Minimax, sau đó thuật toán tìm kiếm sẽ dùng các giá trị vừa nhận được để chọn lựa một nước trong số các nước đi kế tiếp. Bằng cách tối đa hóa cho các cha mẹ MAX và tối thiểu hóa cho các cha mẹ MIN, những giá trị này đi lùi theo đồ thị đến con của trạng thái hiện hành. Sau đó trạng thái hiện hành dùng chúng để tiến hành lựa chọn trong các con của nó. Hình sau trình bày quá trình Minimax trên một không gian trạng thái giả thuyết tính trước bốn lớp.



Hình 4.7 – *Minimax đối với một không gian trạng thái giả định*

Hình 4.8 giới thiệu một ứng dụng của Minimax độ sâu lớp cố định vào trò chơi Tic-tac-toe.



Hình 4.8 – *Minimax hai lớp được áp dụng vào nước đi mở đầu trò chơi Tic-tac-toe*

Ở đây sử dụng một heuristic hơi phức tạp hơn, nó cố đo mức độ tranh chấp trong trò chơi. Heuristic chọn một trạng thái cần đo, tính tất cả các đường thắng mở ra cho MAX, rồi trừ đi tổng số các đường thắng mở ra cho MIN. Giải thuật tìm kiếm sẽ cố gắng tối đa hóa sự chênh lệch (hiệu số) đó. Nếu có một trạng thái bắt buộc thắng cuộc cho MAX, nó sẽ được đánh giá là $+\infty$, còn với trạng thái bắt buộc thắng cuộc cho MIN thì được đánh giá là $-\infty$. Hình 4.8 trình bày heuristic này được áp dụng cho hai mức bắt đầu không gian trạng thái.

Câu hỏi :



Khi một máy tính sử dụng *giải thuật minimax* để chơi cờ, máy tính chơi tốt hơn khi thời gian cho phép để tính toán cho nước cờ kế tiếp là lâu hơn. Hãy giải thích một cách ngắn gọn điều này.

III.3 Thủ tục cắt tỉa alpha – beta (α-β prunning)

Minimax yêu cầu phải có sự phân tích qua hai bước đối với không gian tìm kiếm: Bước đầu truyền xuống đến độ sâu của lớp áp dụng heuristic và bước sau để truyền ngược các giá trị trên cây. Minimax lần theo tất cả các nhánh trong không gian bao gồm cả những nhánh mà một thuật toán thông minh hơn có thể bỏ qua hay tỉa bớt. Các nhà nghiên cứu trong lĩnh vực chơi game đã xây dựng một kỹ thuật tìm kiếm gọi là *cắt tỉa alpha –beta* nhằm nâng cao hiệu quả tìm kiếm trong các bài toán trò chơi hai đối thủ.

Ý tưởng của tìm kiếm alpha – beta rất đơn giản: Thay vì nếu như tìm kiếm toàn bộ không gian đến một độ sâu lớp cố định, tìm kiếm alpha – beta thực hiện theo kiểu tìm kiếm sâu. Có hai giá trị, gọi là alpha và beta được tạo ra trong quá trình tìm kiếm. Giá trị alpha liên quan với các nút MAX và có khuynh hướng không bao giờ giảm. Ngược lại giá trị beta liên quan đến các nút MIN và có khuynh hướng không bao giờ tăng. Giả sử có giá trị alpha của một nút MAX là 6, MAX không cần phải xem xét giá trị truyền ngược nào nhỏ hơn hoặc bằng 6 có liên quan với một nút MIN nào đó bên dưới. Alpha là giá trị thấp nhất mà MAX có thể nhận được sau khi cho rằng MIN cũng sẽ nhận giá trị tốt nhất của nó. Tương tự nếu MIN có giá trị beta là 6 nó cũng không cần xem xét các nút nằm dưới nó có giá trị lớn hơn hoặc bằng 6.

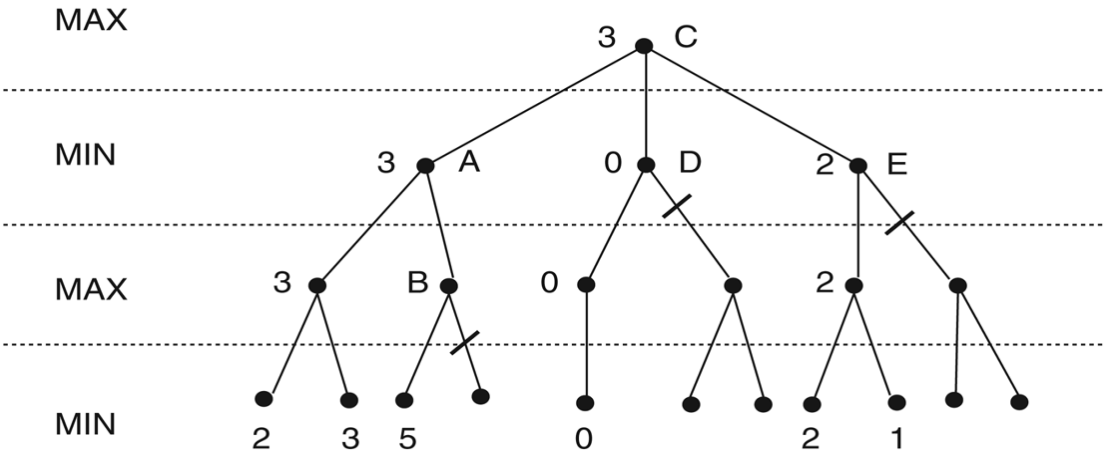
Để bắt đầu thuật toán tìm kiếm alpha – beta, ta đi xuống hết độ sâu lớp theo kiểu tìm kiếm sâu, đồng thời áp dụng đánh giá heuristic cho một trạng thái và tất cả các trạng thái anh em của nó. Giả thuyết tất cả đều là nút MIN. Giá trị tối đa của các nút MIN này sẽ được truyền ngược lên cho nút cha mẹ (là một nút MAX). Sau đó giá trị này được gán cho ông bà của các nút MIN như là một giá trị beta kết thúc tốt nhất. Tiếp theo thuật toán này sẽ đi xuống các nút cháu khác và kết thúc việc tìm kiếm đối với nút cha mẹ của chúng nếu gặp bất kỳ một giá trị nào lớn hơn hoặc bằng giá trị beta này. Quá trình này gọi là *cắt tỉa beta* (β cut). Cách làm tương tự cũng được thực hiện cho việc *cắt tỉa alpha* (α cut) đối với các nút cháu của một nút MAX.

Hai luật cắt tỉa dựa trên các giá trị alpha và beta là:

1. Quá trình tìm kiếm có thể kết thúc bên dưới một nút MIN nào có giá trị beta nhỏ hơn hoặc bằng giá trị alpha của một nút cha MAX bất kỳ của nó.

2. Quá trình tìm kiếm có thể kết thúc bên dưới một nút MAX nào có giá trị alpha lớn hơn hoặc bằng giá trị beta của một nút cha MIN bất kỳ của nó.

Việc cắt tỉa alpha – beta như vậy thể hiện quan hệ giữa các nút ở lớp n và các nút ở lớp n+2 và do quan hệ đó toàn bộ các cây con bắt nguồn ở lớp n+1 đều có thể loại khỏi việc xem xét. Chú ý rằng giá trị truyền ngược thu được hoàn toàn giống như kết quả Minimax, đồng thời tiết kiệm được các bước tìm kiếm một cách đáng kể.



A có β = 3 (Trị nút A sẽ không lớn hơn 3)
B bị cắt tỉa β, vì 5 > 3
C có α = 3 (Trị nút C sẽ không nhỏ hơn 3)
D bị cắt tỉa α, vì 0 < 3
E bị cắt tỉa α, vì 2 < 3
Trị nút C là 3

Hình 4.9 – *Thực hiện giải thuật cắt tỉa alpha – beta*

TỔNG KẾT CHƯƠNG IV: Các heuristic tìm kiếm đã được giới thiệu thông qua các trò chơi đơn giản như trò đố 8 ô, Tic-tac-toe, … và cũng đã được phát triển đến các không gian bài toán phức tạp hơn. Chương này cũng đã trình bày việc áp dụng heuristic cho các trò chơi đối kháng có hai người chơi, dùng cách rút gọn tối thiểu trên độ sâu lớp và cắt tỉa alpha - beta để thực hiện việc tính trước các nước đi và dự đoán hành vi của đối thủ. Việc áp dụng các heuristic này đã làm cho không gian bài toán trở nên ngắn gọn hơn, thời gian tìm kiếm một lời giải có thể chấp nhận được là tối thiểu và quá trình tìm kiếm vì thế cũng trở nên đơn giản hơn khá nhiều. Phần chương V tiếp theo, chúng ta sẽ xem xét các kỹ thuật cao cấp hơn cho việc cài đặt các thuật toán.

IV BÀI TẬP CHƯƠNG IV

IV.1. Xét bài toán trò đố 8 ô như sau:

Start

1	2	3
6	7	
8	5	4

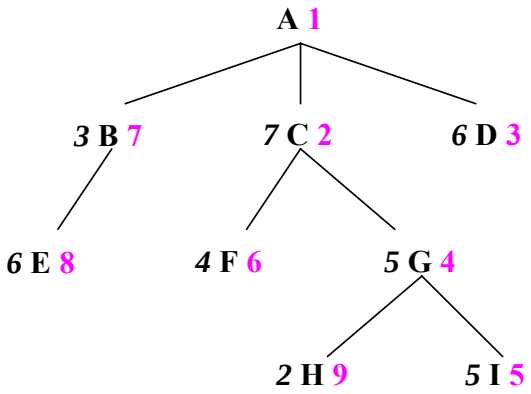
Goal

1	2	3
8		4
7	6	5

Dùng các hàm lượng giá heuristic sau, hãy triển khai không gian trạng thái của bài toán theo giải thuật leo núi đến mức 5:

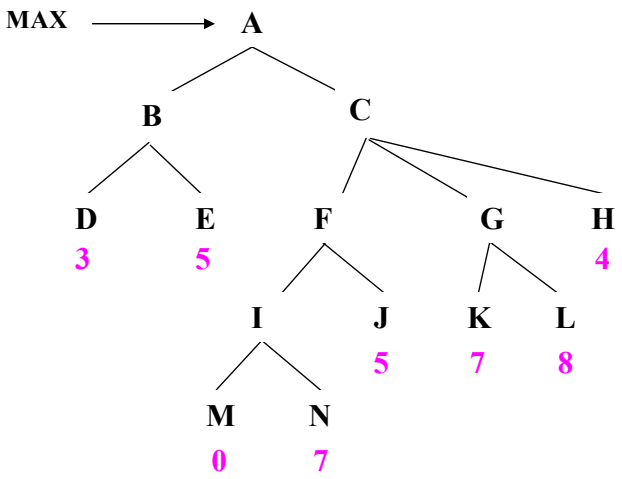
a) h1 = số lượng các vị trí sai khác so với trạng thái goal.
b) h2 = tổng số độ dời ngắn nhất của các ô về vị trí đúng (khoảng cách Manhattan)

IV.2. Trong cây tìm kiếm dưới đây, mỗi nút có 2 giá trị đi kèm: giá trị bên trái của nút (in nghiêng) thể hiện giá trị heuristic của nút, và giá trị bên phải nút thể hiện thứ tự nút được duyệt qua. Với mỗi chiến lược tìm kiếm dưới đây, hãy viết danh sách thứ tự các nút được duyệt, so sánh và cho biết ta đã dùng giải thuật tìm kiếm nào trên cây :



a) Tìm kiếm rộng BFS
b) Tìm kiếm sâu DFS
c) Tìm kiếm tốt nhất đầu tiên
d) Tìm kiếm leo núi

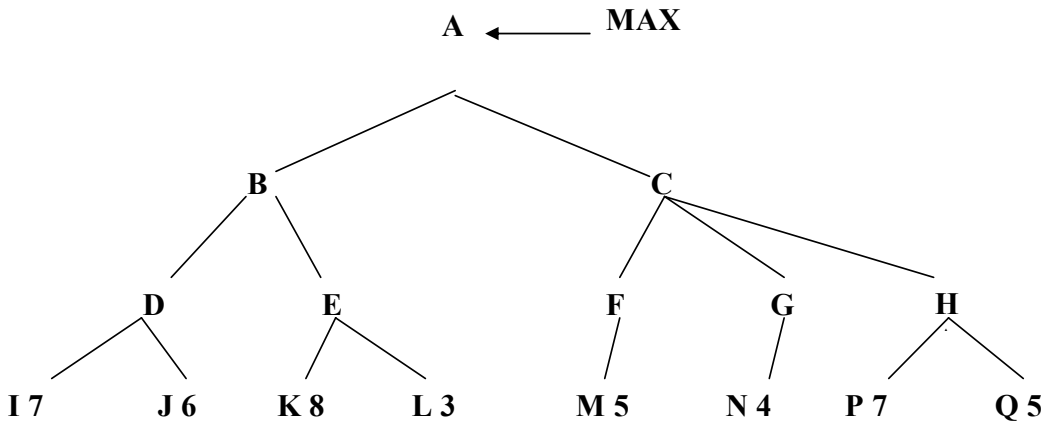
IV.3. Thực hiện giải thuật Minimax trên cây sau đây:



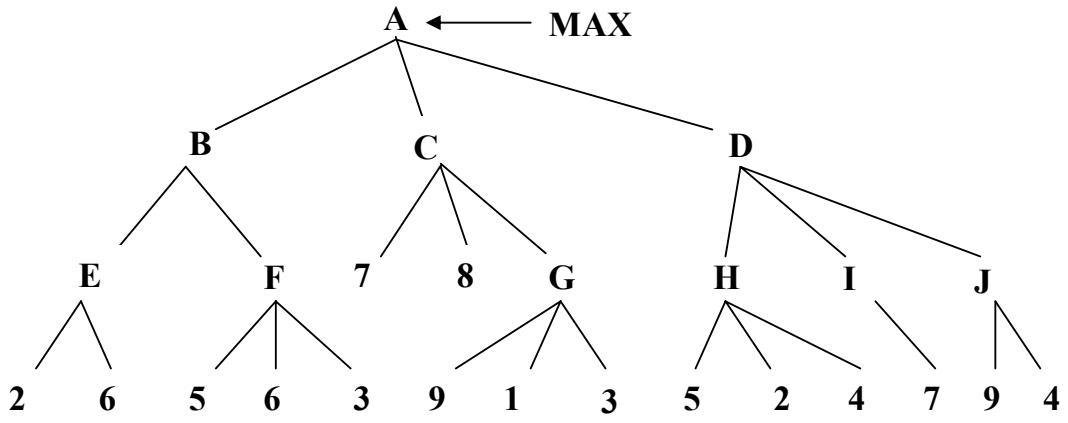
Sẽ có gì khác biệt nếu như ta dùng giải thuật cắt tỉa alpha – beta để định trị nút gốc cho cây?

IV.4. Hãy áp dụng giải thuật cắt tỉa alpha-beta cho các cây sau đây. Cho biết các nhánh được cắt là alpha-cut hay beta-cut và giá trị nút gốc sau khi định trị:

a)



b)



Chương IV 63
TÌM KIẾM HEURISTIC 63
I. MỞ ĐẦU 64
II. THUẬT TOÁN TÌM KIẾM HEURISTIC 67
II.1. Tìm kiếm leo núi (Hill climbing – Pearl 1984) 67
II.2. Tìm kiếm tốt nhất đầu tiên (Best – first – search) 67
II.3. Cài đặt hàm đánh giá heuristic (heuristic evaluation function) 69
II.4. Tính khả chấp, tính đơn nhất và khả năng cung cấp thông tin của heuristic 70
III. SỬ DỤNG HEURISTIC TRONG CÁC TRÒ CHƠI 72
III.1. Thủ tục minimax 72
III.2. Áp dụng minimax đến độ sâu lớp cố định 73
III.3. Thủ tục cắt tỉa alpha – beta (α-β prunning) 75
BÀI TẬP CHƯƠNG IV 77